MODUL PRAKTIKUM 6
PEMBANGUNAN ANTARMUKA BERBASIS WIDGET

6.1. Topik Bahasan

Modul ini dilaksanakan pada pertemuan ke-6 sesuai Rencana Pembelajaran Semester mata kuliah Pemrograman IV dan membebani CPMK-02, dengan cakupan topik sebagai berikut:

- Perbedaan `StatelessWidget` dan `StatefulWidget`.
- Tata letak menggunakan `Row`, `Column`, `Container`, dan `Expanded`.
- *Widget input* dan widget penyajian data.
- Penyajian daftar data menggunakan `ListView.builder`.
- Penerapan tema aplikasi melalui `ThemeData`.
- Navigasi antarhalaman dan pembuatan layar daftar serta detail pengiriman.

6.2. Capaian Pembelajaran

Mahasiswa mampu membangun antarmuka aplikasi mobile sesuai rancangan yang ditetapkan (CPMK-02).

6.3. Indikator Ketercapaian Pembelajaran

Indikator penilaian pertemuan ke-6 menurut Rencana Pembelajaran Semester adalah:

- Ketepatan penyusunan widget sesuai rancangan
- Keberfungsian navigasi antarhalaman

Indikator tersebut dinyatakan tercapai apabila bukti berikut terlihat pada hasil praktikum:

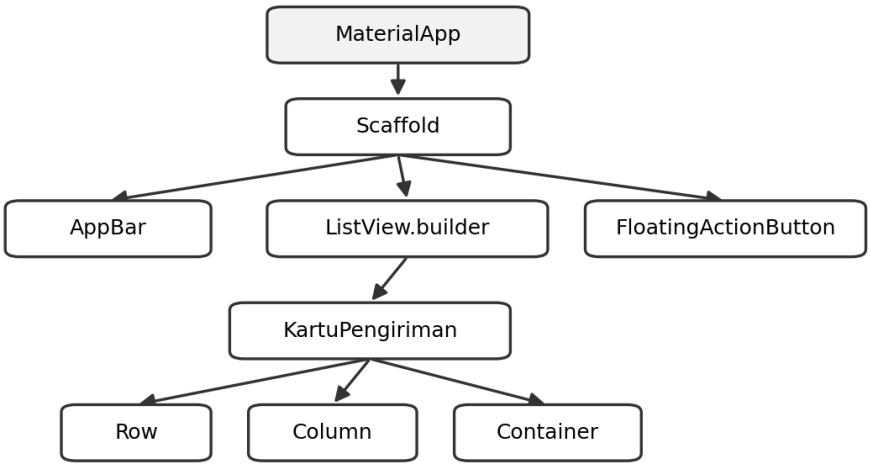
- Layar daftar dan layar detail pengiriman tampil sesuai rancangan.
- Navigasi antarhalaman berfungsi, termasuk perpindahan kembali ke layar sebelumnya.
- Tampilan tetap terbaca pada *browser* maupun emulator dengan lebar layar yang berbeda.
- Struktur widget tersusun rapi dan dipecah menjadi widget terpisah yang dapat digunakan ulang.

6.4. Teori Pendukung

Widget sebagai Satuan Penyusun Antarmuka

Pada Flutter, seluruh elemen antarmuka adalah widget, termasuk hal yang tidak terlihat seperti jarak antar elemen dan pengatur tata letak. Widget disusun bertingkat membentuk *widget tree*. Prinsip ini disebut komposisi: alih-alih mengatur banyak

properti pada satu komponen besar, antarmuka dibangun dengan menggabungkan banyak widget kecil yang masing-masing memiliki satu tanggung jawab.



Gambar 6. Widget tree layar daftar pengiriman

StatelessWidget dan StatefulWidget

`StatelessWidget` dipakai untuk bagian antarmuka yang tampilannya sepenuhnya ditentukan oleh data yang diterimanya dan tidak berubah sendiri, misalnya kartu penyaji data pengiriman. `StatefulWidget` dipakai ketika bagian antarmuka menyimpan keadaan yang dapat berubah selama widget hidup, misalnya daftar yang dapat difilter atau formulir yang sedang diisi. `StatefulWidget` terdiri atas dua kelas: kelas widget itu sendiri dan kelas `State` yang menyimpan datanya.

Pada praktiknya `StatelessWidget` dipilih lebih dahulu, sedangkan `StatefulWidget` dipakai hanya ketika widget perlu menyimpan keadaan. Semakin sedikit widget yang menyimpan keadaan, semakin mudah aplikasi dipelihara.

Widget Tata Letak

- `Column` menyusun anak-anaknya secara vertikal, `Row` secara horizontal. Keduanya diatur melalui `mainAxisAlignment` dan `crossAxisAlignment`.
- `Container` membungkus satu anak sekaligus menyediakan padding, margin, warna, batas, dan sudut membulat.
- `Padding` dan `SizedBox` mengatur jarak. `SizedBox` juga dipakai untuk menetapkan ukuran tetap.
- `Expanded` dan `Flexible` membagi ruang sisa pada `Row` atau `Column` sehingga tampilan tidak meluap ketika lebar layar berubah.
- `Stack` menumpuk widget, dipakai misalnya untuk menempatkan label status di atas gambar.

- `Card` dan `ListTile` menyediakan bentuk baku penyajian data yang sudah mengikuti kaidah Material Design.

Tabel 7. Widget tata letak yang dipakai pada layar daftar dan detail pengiriman

Widget	Fungsi	Dipakai pada
`Scaffold`	Kerangka halaman beserta AppBar dan badan	Kedua layar
`Column`	Menyusun anak secara vertikal	Isi kartu dan layar detail
`Row`	Menyusun anak secara horizontal	Baris kartu pengiriman
`Expanded`	Membagi ruang sisa agar tidak meluap	Kolom teks pada kartu
`Container`	Padding, warna, batas, dan sudut membulat	Label status
`ListView.builder`	Daftar panjang yang dibangun sesuai kebutuhan	Layar daftar
`Card` dan `ListTile`	Bentuk baku penyajian data Material Design	Kartu pengiriman
`SingleChildScrollView`	Menggulir isi yang lebih tinggi dari layar	Layar detail

Penyajian Daftar Data

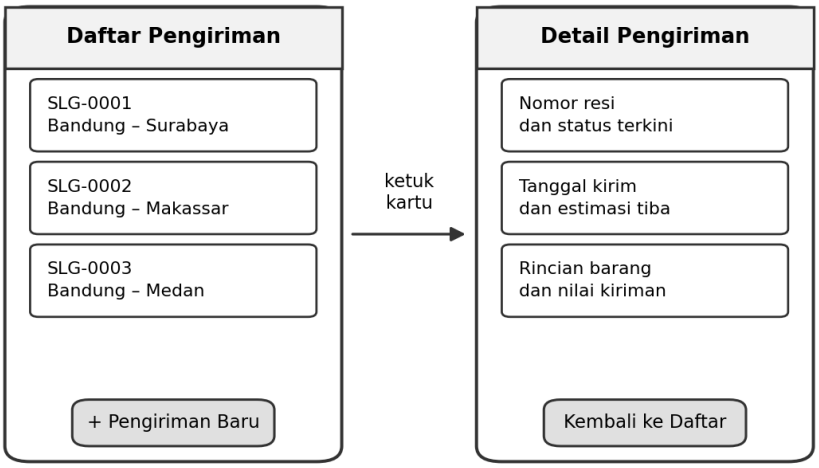
Untuk daftar data yang jumlahnya banyak atau tidak diketahui sejak awal, `ListView.builder` merupakan pilihan yang tepat karena hanya membangun elemen yang terlihat di layar. Cara ini jauh lebih hemat dibandingkan `ListView` biasa yang membangun seluruh elemen sekaligus. Properti `itemCount` menentukan jumlah data dan `itemBuilder` menentukan cara satu baris data digambarkan.

Tema Aplikasi

Tema ditetapkan satu kali pada `MaterialApp` melalui `ThemeData`. Dengan `ColorScheme.fromSeed`, seluruh warna aplikasi diturunkan dari satu warna dasar sehingga tampilan konsisten. Warna dan gaya teks kemudian diacu melalui `Theme.of(context)`, bukan dituliskan berulang pada tiap widget. Pendekatan ini memudahkan penggantian identitas visual pada tahap proyek.

Navigasi Antarhalaman

Flutter mengelola halaman sebagai tumpukan (stack). `Navigator.push` menambahkan halaman baru di atas tumpukan, sedangkan `Navigator.pop` mengeluarkan halaman teratas sehingga pengguna kembali ke halaman sebelumnya. Data dapat dikirim ke halaman tujuan melalui konstruktor halaman tersebut, dan dapat dikembalikan ke halaman asal melalui argumen `Navigator.pop`.



Gambar 7. Rancangan layar daftar dan layar detail pengiriman beserta navigasinya

6.5. Alat Pendukung

Perangkat keras, perangkat lunak, dan bahan yang diperlukan pada modul ini adalah:

- Flutter SDK dan Dart SDK.
- Visual Studio Code beserta ekstensi Flutter dan Dart.
- Google Chrome atau Microsoft Edge.
- Figma untuk membuat rancangan antarmuka (*wireframe*) sebelum menulis kode.
- Flutter DevTools untuk memeriksa widget tree.
- Repositori GitHub pribadi praktikan (satu repositori untuk seluruh modul praktikum).

6.6. Langkah-langkah Praktikum

A. Penyiapan dan Perancangan Antarmuka

1. Buka proyek `silog_app` melalui menu File > Open Folder pada Visual Studio Code, lalu buka terminal terintegrasi melalui menu Terminal > New Terminal. Buat cabang kerja modul ini pada terminal dengan perintah `git checkout -b praktikum/modul-06`, kemudian pastikan cabang aktif melalui `git branch`.
2. Buat rancangan wireframe dua layar pada Figma melalui menu File > New design file, yaitu layar daftar pengiriman dan layar detail pengiriman, lalu simpan sebagai gambar melalui menu File > Export untuk dilampirkan pada jurnal.
3. Buat direktori `lib/tampilan` dan `lib/widget` melalui panel Explorer: klik kanan direktori `lib`, pilih New Folder, lalu ketik nama tiap direktori.

B. Penyiapan Data Contoh

1. Buat direktori `lib/data` melalui panel Explorer apabila belum ada (klik kanan direktori `lib`, pilih New Folder, ketik `data`), kemudian buat berkas

`data_contoh.dart` di dalamnya melalui New File. Ketik kode berikut, lalu simpan melalui `Ctrl+S`.

Kode Program 6.1 Berkas lib/data/data_contoh.dart

```dart
import '../model/barang.dart';
import '../model/gudang.dart';
import '../model/pengiriman.dart';

final gudangBandung = Gudang(
    kode: 'GD-BDG', nama: 'Gudang Utama Bandung', kota: 'Bandung');

final List<Pengiriman> dataPengiriman = [
  Pengiriman(
    resi: 'SLG-2026-0001',
    asal: gudangBandung,
    kotaTujuan: 'Surabaya',
    barang: Barang(kode: 'B-1024', nama: 'Sepatu Lari Pria', hargaSatuan: 450000),
    jumlah: 10,
    tanggalKirim: DateTime(2026, 9, 8),
    status: 'Dalam Perjalanan',
  ),
  Pengiriman(
    resi: 'SLG-2026-0002',
    asal: gudangBandung,
    kotaTujuan: 'Makassar',
    barang: Barang(kode: 'B-2048', nama: 'Kaos Katun', hargaSatuan: 85000),
    jumlah: 120,
    tanggalKirim: DateTime(2026, 9, 9),
    status: 'Menunggu Pickup',
  ),
  Pengiriman(
    resi: 'SLG-2026-0003',
    asal: gudangBandung,
    kotaTujuan: 'Medan',
    barang: Barang(kode: 'B-3072', nama: 'Tas Ransel', hargaSatuan: 275000),
    jumlah: 35,
    tanggalKirim: DateTime(2026, 9, 10),
    status: 'Diterima',
  ),
];
```

C. Widget Kartu Pengiriman

1. Buat berkas `lib/widget/kartu_pengiriman.dart` melalui panel Explorer: klik kanan direktori `lib/widget`, pilih New File, lalu ketik nama berkas `kartu_pengiriman.dart`. Ketik kode berikut, kemudian simpan melalui `Ctrl+S`.

Kode Program 6.2 Widget kartu penyaji data pengiriman

```dart
import 'package:flutter/material.dart';
import '../model/pengiriman.dart';

class KartuPengiriman extends StatelessWidget {
  final Pengiriman pengiriman;
  final VoidCallback onTap;

  const KartuPengiriman({
    super.key,
    required this.pengiriman,
    required this.onTap,
  });
```

```
  Color _warnaStatus(BuildContext context) {
    switch (pengiriman.status) {
      case 'Diterima':
        return Colors.green;
      case 'Dalam Perjalanan':
        return Colors.orange;
      default:
        return Theme.of(context).colorScheme.outline;
    }
  }

  @override
  Widget build(BuildContext context) {
    final teks = Theme.of(context).textTheme;

    return Card(
      margin: const EdgeInsets.symmetric(horizontal: 12, vertical: 6),
      child: InkWell(
        onTap: onTap,
        borderRadius: BorderRadius.circular(12),
        child: Padding(
          padding: const EdgeInsets.all(12),
          child: Row(
            children: [
              CircleAvatar(
                backgroundColor: _warnaStatus(context).withOpacity(0.15),
                child: Icon(Icons.inventory_2, color: _warnaStatus(context)),
              ),
              const SizedBox(width: 12),
              Expanded(
                child: Column(
                  crossAxisAlignment: CrossAxisAlignment.start,
                  children: [
                    Text(pengiriman.resi, style: teks.titleMedium),
                    const SizedBox(height: 2),
                    Text('${pengiriman.barang.nama} x ${pengiriman.jumlah}',
                        style: teks.bodyMedium),
                    const SizedBox(height: 2),
                    Text(
                      '${pengiriman.asal.kota} \u2192 ${pengiriman.kotaTujuan}',
                      style: teks.bodySmall,
                    ),
                  ],
                ),
              ),
              Container(
                padding:
                    const EdgeInsets.symmetric(horizontal: 8, vertical: 4),
                decoration: BoxDecoration(
                  color: _warnaStatus(context).withOpacity(0.12),
                  borderRadius: BorderRadius.circular(20),
                ),
                child: Text(
                  pengiriman.status,
                  style: teks.labelSmall?.copyWith(color: _warnaStatus(context)),
                ),
              ),
            ],
          ),
        ),
      ),
    );
  }
}
```

D. Layar Daftar Pengiriman

1. Buat berkas `lib/tampilan/daftar_pengiriman_page.dart` melalui panel Explorer: klik kanan direktori `lib/tampilan`, pilih New File, lalu ketik nama berkas `daftar_pengiriman_page.dart`. Ketik kode berikut, kemudian simpan melalui `Ctrl+S`.

Kode Program 6.3 Layar daftar pengiriman

```
import 'package:flutter/material.dart';
import '../data/data_contoh.dart';
import '../widget/kartu_pengiriman.dart';
import 'detail_pengiriman_page.dart';

class DaftarPengirimanPage extends StatelessWidget {
  const DaftarPengirimanPage({super.key});

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(
        title: const Text('Daftar Pengiriman'),
        actions: [
          IconButton(
            icon: const Icon(Icons.search),
            onPressed: () {},
            tooltip: 'Cari resi',
          ),
        ],
      ),
      body: ListView.builder(
        itemCount: dataPengiriman.length,
        itemBuilder: (context, index) {
          final item = dataPengiriman[index];
          return KartuPengiriman(
            pengiriman: item,
            onTap: () {
              Navigator.push(
                context,
                MaterialPageRoute(
                  builder: (_) => DetailPengirimanPage(pengiriman: item),
                ),
              );
            },
          );
        },
      ),
      floatingActionButton: FloatingActionButton.extended(
        onPressed: () {},
        icon: const Icon(Icons.add),
        label: const Text('Pengiriman Baru'),
      ),
    );
  }
}
```

E. Layar Detail Pengiriman

1. Buat berkas `lib/tampilan/detail_pengiriman_page.dart` melalui panel Explorer: klik kanan direktori `lib/tampilan`, pilih New File, lalu ketik nama berkas `detail_pengiriman_page.dart`. Ketik kode berikut, kemudian simpan melalui `Ctrl+S`.

Kode Program 6.4 Layar detail pengiriman

```
import 'package:flutter/material.dart';
import '../model/pengiriman.dart';

class DetailPengirimanPage extends StatelessWidget {
  final Pengiriman pengiriman;

  const DetailPengirimanPage({super.key, required this.pengiriman});

  Widget _baris(String label, String nilai) {
    return Padding(
      padding: const EdgeInsets.symmetric(vertical: 6),
      child: Row(
        crossAxisAlignment: CrossAxisAlignment.start,
        children: [
          SizedBox(
            width: 130,
            child: Text(label, style: const TextStyle(color: Colors.grey)),
          ),
          const Text(': '),
          Expanded(child: Text(nilai)),
        ],
      ),
    );
  }

  @override
  Widget build(BuildContext context) {
    final tgl = pengiriman.tanggalKirim;
    return Scaffold(
      appBar: AppBar(title: Text(pengiriman.resi)),
      body: SingleChildScrollView(
        padding: const EdgeInsets.all(16),
        child: Column(
          crossAxisAlignment: CrossAxisAlignment.start,
          children: [
            Text('Informasi Pengiriman',
                style: Theme.of(context).textTheme.titleLarge),
            const Divider(height: 24),
            _baris('Nomor Resi', pengiriman.resi),
            _baris('Status', pengiriman.status),
            _baris('Gudang Asal', pengiriman.asal.nama),
            _baris('Kota Tujuan', pengiriman.kotaTujuan),
            _baris('Tanggal Kirim',
                '${tgl.day}/${tgl.month}/${tgl.year}'),
            const SizedBox(height: 16),
            Text('Rincian Barang',
                style: Theme.of(context).textTheme.titleLarge),
            const Divider(height: 24),
            _baris('Kode Barang', pengiriman.barang.kode),
            _baris('Nama Barang', pengiriman.barang.nama),
            _baris('Jumlah', '${pengiriman.jumlah} ${pengiriman.barang.satuan}'),
            _baris('Nilai Barang',
                'Rp${pengiriman.nilaiBarang.toStringAsFixed(0)}'),
            const SizedBox(height: 24),
            SizedBox(
```

```
              width: double.infinity,
              child: FilledButton.icon(
                onPressed: () => Navigator.pop(context),
                icon: const Icon(Icons.arrow_back),
                label: const Text('Kembali ke Daftar'),
              ),
            ),
          ],
        ),
      ),
    );
  }
}
```

F. Penerapan Tema dan Penyatuan Aplikasi

1. Buka berkas `lib/main.dart` melalui panel Explorer, ganti seluruh isinya dengan kode berikut agar `DaftarPengirimanPage` menjadi halaman awal dan tema aplikasi ditetapkan, lalu simpan melalui `Ctrl+S`.

Kode Program 6.5 Penyesuaian lib/main.dart

```
import 'package:flutter/material.dart';
import 'tampilan/daftar_pengiriman_page.dart';

void main() => runApp(const SiLogApp());

class SiLogApp extends StatelessWidget {
  const SiLogApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      title: 'SiLog',
      debugShowCheckedModeBanner: false,
      theme: ThemeData(
        useMaterial3: true,
        colorScheme: ColorScheme.fromSeed(seedColor: const Color(0xFF1B5E20)),
        cardTheme: CardTheme(
          elevation: 1,
          shape: RoundedRectangleBorder(
            borderRadius: BorderRadius.circular(12),
          ),
        ),
        appBarTheme: const AppBarTheme(centerTitle: true),
      ),
      home: const DaftarPengirimanPage(),
    );
  }
}
```

2. Jalankan aplikasi melalui `flutter run -d chrome`.
3. Klik salah satu kartu pengiriman untuk memastikan perpindahan ke layar detail berfungsi, lalu kembali melalui tombol panah pada `AppBar`.
4. Buka Developer Tools browser melalui `F12`, aktifkan mode perangkat melalui `Ctrl+Shift+M`, pilih profil ponsel, kemudian periksa apakah terjadi *overflow*.

5. Buka Command Palette melalui `Ctrl+Shift+P`, jalankan perintah `Flutter: Open DevTools` lalu pilih Open DevTools in Browser, buka tab Flutter Inspector, telusuri widget tree layar daftar, dan lampirkan tangkapan layarnya pada jurnal.

G. Tugas Praktikum

1. Tambahkan `TabBar` dengan tiga tab pada layar daftar: Semua, Dalam Perjalanan, dan Diterima. Tiap tab menampilkan daftar yang telah difilter.
2. Tambahkan widget ringkasan pada bagian atas layar daftar yang menampilkan jumlah pengiriman dan total nilai barang.
3. Pastikan seluruh widget yang dapat dijadikan `const` telah ditandai `const`, lalu jelaskan manfaatnya pada jurnal.

H. Luaran yang Dinilai

Bentuk penilaian modul ini adalah tugas praktikum dengan kriteria kesesuaian antarmuka dengan rancangan dan keberfungsian komponennya. jurnal praktikum individu diunggah ke lms ulbi paling lambat 5x24 jam setelah sesi praktikum. Agar kriteria tersebut dapat dinilai, praktikan menyerahkan luaran berikut:

- Wireframe layar daftar dan layar detail pengiriman dari Figma.
- Berkas pada direktori `lib/tampilan` dan `lib/widget`.
- Tangkapan layar kedua layar pada lebar jendela penuh dan pada lebar sekitar 400 piksel, tanpa overflow.
- Bukti navigasi berfungsi berupa tangkapan layar berurutan: daftar, detail, lalu kembali ke daftar.
- Tangkapan layar widget tree pada Widget Inspector.

6.7. Panduan Pengumpulan Jurnal Hasil Kegiatan Praktikum

Tugas praktikum modul ini bersifat individu. Setiap praktikan menyusun dan mengunggah sendiri jurnalnya ke LMS ULBI paling lambat 5 x 24 jam setelah sesi praktikum berakhir, berupa berkas PDF bernama `NIM_Nama_Modul-XX.pdf`, disertai tautan repositori GitHub pribadi pada cabang `praktikum/modul-XX`.

Sistematika jurnal:

1. Identitas: nama, NIM, kelas, nomor dan judul modul, serta tanggal praktikum.
2. Tujuan praktikum.
3. Alat dan bahan yang digunakan beserta versinya.
4. Langkah kerja, disertai tangkapan layar tiap tahap penting.
5. Kode program beserta penjelasan singkatnya.
6. Hasil eksekusi dan analisisnya, termasuk *error* yang ditemui beserta penyelesaiannya.
7. Jawaban seluruh butir Tugas Praktikum.
8. Kesimpulan dan kendala.

9. Tautan repositori beserta tangkapan layar riwayat *commit*.

Tangkapan layar wajib memperlihatkan jendela browser atau emulator secara utuh beserta kode programnya, dan memuat nama serta NIM praktikan. Praktikan yang tidak mengumpulkan jurnal memperoleh nilai nol; keterlambatan dan ketidaksesuaian hasil lainnya dikenai pengurangan nilai sesuai kebijakan dosen pengampu.

Seluruh butir pada sub bab Luaran yang Dinilai wajib dilampirkan pada jurnal.

6.8. Hasil Capaian Pembelajaran Praktikum

Bentuk dan kriteria penilaian mengikuti Rencana Pembelajaran Semester pertemuan ke-6: Tugas praktikum: kesesuaian antarmuka dengan rancangan dan keberfungsian komponennya. Jurnal praktikum individu diunggah ke LMS ULBI paling lambat 5x24 jam setelah sesi praktikum.

Tugas praktikum modul ini bersifat individu dan dinilai dari jurnal praktikum masing-masing praktikan.

Bobotnya 2% dari nilai akhir mata kuliah pada komponen evaluasi Praktikum (Aktivitas Partisipatif) sebesar 15% dari nilai akhir, diukur dari kelengkapan dan mutu luaran pada sub bab Luaran yang Dinilai terhadap indikator penilaian modul ini.